

Эссе «Что такое Domain-Driven Design?»

Понятие предметной области

Предметная область (domain) — это часть реального мира, для которой создаётся программная система: совокупность понятий, правил, процессов и участников, существующих независимо от того, пишется ли для них программа. Например, для интернет-магазина предметной областью является розничная торговля: товары, заказы, оплаты, доставка, скидки, возвраты; для банковского приложения — счета, платежи, проценты по вкладам, кредитные лимиты. Предметная область осмысленна сама по себе — эксперт в ней (продавец, бухгалтер, логист) может о ней рассуждать, даже никогда не видев кода программы.

Что такое Domain-Driven Design

Domain-Driven Design (предметно-ориентированное проектирование, DDD) — подход к проектированию ПО, при котором структура кода строится не вокруг технических удобств (баз данных, фреймворков), а вокруг модели предметной области — понятий и правил, которыми реально оперируют пользователи и эксперты. Термин ввёл Эрик Эванс в книге «Domain-Driven Design: Tackling Complexity in the Heart of Software» (2003). Идея в том, что самая сложная и изменчивая часть системы — не технологии, а бизнес-логика. Если правила предметной области спрятаны внутри функций, работающих напрямую с таблицами базы данных или примитивными типами, со временем становится всё труднее понимать, что делает программа, а любое изменение требования превращается в риск что-то сломать.

Зачем нужно предметно-ориентированное проектирование

Назначение DDD — сделать так, чтобы структура кода отражала структуру знаний о предметной области, а не только техническую реализацию. Для этого DDD предлагает несколько ключевых идей:

- **Единый язык (Ubiquitous Language).** Разработчики и эксперты предметной области договариваются об общем словаре терминов и используют его одинаково — как в разговоре с заказчиком, так и в именах классов, функций и переменных в коде. Если эксперт называет что-то «заказом», в коде тоже должен быть класс `Order`, а не абстрактная `Record` или `Item`.

- **Модель предметной области.** Код строится вокруг объектов, которые отражают реальные понятия предметной области — их свойства, поведение и связи между ними, — а не вокруг структуры хранения данных. Модель должна воспроизводить не только данные, но и правила: например, «нельзя списать со счёта больше, чем на нём есть денег» — это не проверка где-то в интерфейсе, а неотъемлемое поведение самого объекта «Счёт».
- **Ограниченный контекст (Bounded Context).** Одно и то же слово может означать разное в разных частях большой системы: «Товар» в контексте склада — это единица учёта с адресом хранения, а «Товар» в контексте маркетинга — это карточка с описанием и фотографиями. DDD признаёт, что не нужно натягивать одну общую модель на всю систему целиком, а вместо этого явно выделяет границы, внутри которых модель и её словарь остаются согласованными.
- **Разделение сложной и простой логики.** DDD различает «богатую» модель предметной области (с поведением и правилами) и вспомогательные технические слои (хранение данных, отправка сообщений, отображение интерфейса), не позволяя техническим деталям просачиваться внутрь модели и исказить её.

Как знания о предметной области отражаются в структуре программы

На практике это означает, что структура классов и модулей программы должна быть узнаваема экспертом предметной области. Если посмотреть на список классов системы, названия должны напоминать словарь этой области, а не словарь баз данных или веб-фреймворков. Связи между классами тоже должны отражать реальные связи между понятиями: если в реальности заказ состоит из позиций, а каждая позиция ссылается на товар, то и в коде должна быть примерно такая же структура — объект `Order`, содержащий список объектов `OrderLine`, каждый из которых ссылается на `Product`. Более того, правила предметной области (например, «сумма заказа не может быть отрицательной» или «скидка не может превышать стоимость товара») должны быть выражены как поведение этих же объектов, а не разбросаны по случайным местам интерфейса или обработчиков запросов.

Собственный пример: система учёта посещений спортивной секции

В качестве собственного примера рассмотрю небольшую предметную область — учёт занятий в спортивной секции (например, в кружке борьбы или в бассейне), похожую по духу на пример магазина из конспекта занятия по ООП, но из другой сферы.

Объекты предметной области:

- **Спортсмен (Athlete)** — человек, посещающий занятия; свойства: имя, дата рождения, группа.

- **Тренер (Coach)** — ведущий занятия; свойства: имя, специализация (виды спорта, которые он может вести).
- **Группа (Group)** — набор спортсменов, занимающихся вместе под руководством одного тренера; свойства: название, возрастная категория, расписание.
- **Занятие (Session)** — конкретное проведённое занятие группы в определённый день и время; свойства: дата, длительность, список присутствовавших спортсменов.
- **Абонемент (Membership)** — оплаченное право спортсмена посещать занятия в течение периода; свойства: срок действия, количество оплаченных занятий.

Операции над этими объектами тоже взяты из реальной практики секции, а не придуманы под удобство хранения в базе данных:

- у объекта `Session` есть операция `mark_attendance(athlete)` — отметить присутствие спортсмена;
- у объекта `Membership` есть операция `can_attend()` — проверка, осталось ли у спортсмена право посетить ещё одно занятие по текущему абонементу, и операция `consume_visit()`, которая списывает одно занятие после фактического посещения;
- у объекта `Group` есть операция `schedule_session(date)` — назначить новое занятие по расписанию группы.

Связи между объектами тоже отражают реальную структуру предметной области: спортсмен состоит в одной группе, группу ведёт один тренер, у группы есть много занятий, у занятия — список присутствовавших спортсменов, а у каждого спортсмена — один активный абонемент.

Ключевая мысль DDD на этом примере: правило «спортсмен не может быть отмечен присутствующим на занятии, если у него закончился абонемент» должно быть частью поведения объекта `Session` или `Membership` (например, внутри метода `mark_attendance` при отметке присутствия), а не отдельной проверкой, спрятанной где-то в коде экрана администратора секции. Тогда это правило будет соблюдаться одинаково независимо от того, откуда оно вызывается — из мобильного приложения, из веб-панели администратора или из будущей интеграции с турникетом на входе, — потому что оно живёт внутри самой модели предметной области, а не дублируется в каждом месте, где эта модель используется.

Связь с материалами занятия по ООП

Domain-Driven Design логично продолжает идеи объектно-ориентированного подхода, разобранные на занятии по ООП: предметная область представляется как совокупность взаимодействующих объектов, у каждого объекта есть состояние и операции, а инкапсуляция скрывает детали реализации за понятным интерфейсом. Разница в

акценте: если ООП — это набор технических инструментов (классы, наследование, полиморфизм), то DDD — это дисциплина того, как именно использовать эти инструменты, чтобы модель в коде оставалась верной моделью реального мира, а не превращалась в удобную, но оторванную от смысла структуру данных.